# EXPERIMENT 6

### K-Means and Hierarchical Clustering

## AIM

To implement **K-Means Clustering and Hierarchical Clustering** on a dataset and analyze the grouping of data points based on similarity.

## THEORY

Clustering is an unsupervised learning technique used to group similar data points.

### K-Means Clustering

- Divides data into **K clusters**
- Minimizes distance between points and cluster centroid

### Hierarchical Clustering

- Builds a tree-like structure (dendrogram)
- Two types:
    - Agglomerative (bottom-up)
    - Divisive (top-down)

## Mathematical Formulation

### K-Means Objective Function:

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where:

- $Ci$Ci = cluster
- $\mu i$μi = centroid

## DATASET SOURCE

Dataset: **Mall Customers Dataset**
Source: Kaggle
Link: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial

## DATASET DESCRIPTION

This dataset contains customer information used for segmentation.

**Dataset Details:**

- Total Instances: 200
- Features: 5
- Important Features Used:
    - Annual Income
    - Spending Score
- Dataset Type: Numerical + Categorical
- Domain: Marketing

## ALGORITHM LIMITATIONS

**K-Means:**

- Needs predefined K
- Sensitive to outliers
- Assumes spherical clusters

**Hierarchical Clustering:**

- Computationally expensive
- Not suitable for large datasets
- Difficult to choose optimal clusters

## CODE IMPLEMENTATION

**Step 1: Import Libraries**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
```

**Step 2: Upload and Load Dataset**

```
from google.colab import files
uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])
df.head()
```

**Step 3: Data Understanding**

```
df.info()
df.describe()
df.isnull().sum()
```

**Step 4: Select Features**

```
# Use important features
X = df.iloc[:, [3, 4]]   # Annual Income & Spending Score
```

**Step 5: Feature Scaling**

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

**Step 6: Elbow Method (Find Optimal K)**

```
wcss = []

for i in range(1, 11):
  kmeans = KMeans(n_clusters=i, random_state=42)
  kmeans.fit(X_scaled)
  wcss.append(kmeans.inertia_)

plt.plot(range(1, 11), wcss)
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```

**Step 7: Apply K-Means**

```
kmeans = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)
```

**Step 8: Visualize Clusters**

```
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_kmeans, cmap='viridis')
plt.title("K-Means Clustering")
plt.show()
```

**Step 9: Hierarchical Clustering (Dendrogram)**

```
linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(8,5))
dendrogram(linked)
plt.title("Dendrogram")
plt.show()
```

**PERFORMANCE ANALYSIS**

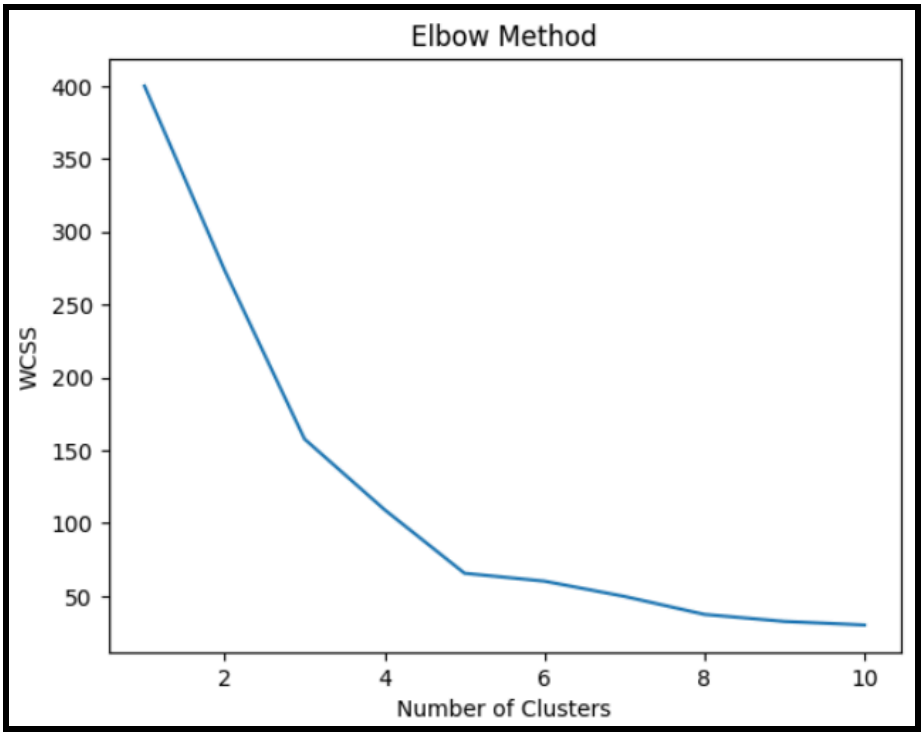
- Elbow method helps find optimal K
- K-Means groups customers effectively
- Hierarchical clustering provides cluster hierarchy
- Dendrogram helps visualize relationships

**<u>OUTPUT</u>**

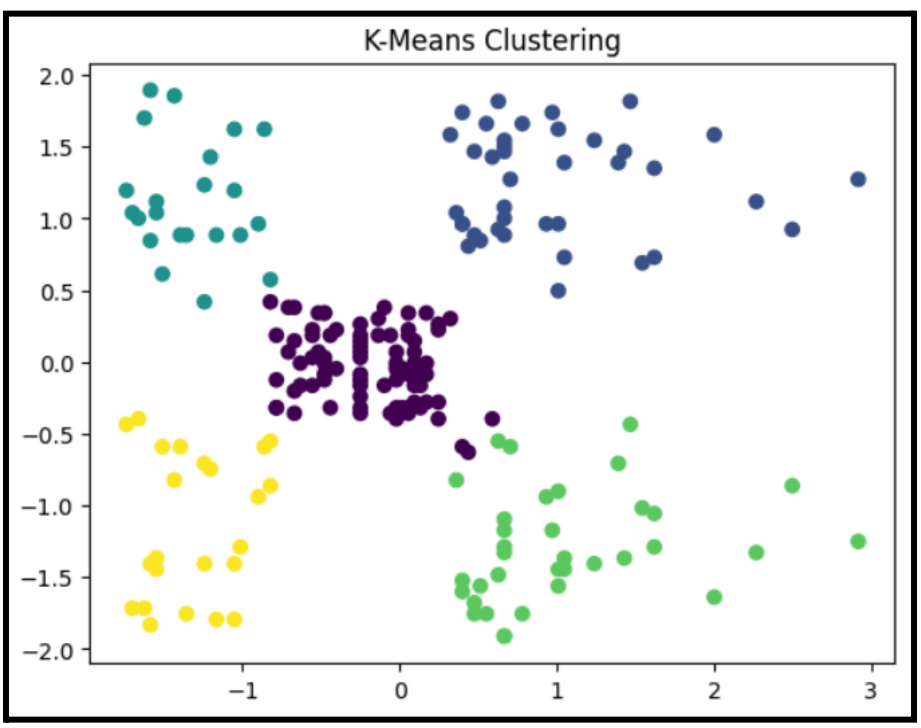
- Optimal clusters identified using Elbow method
- K-Means created clear clusters
- Dendrogram showed hierarchical grouping

**Screenshots:**

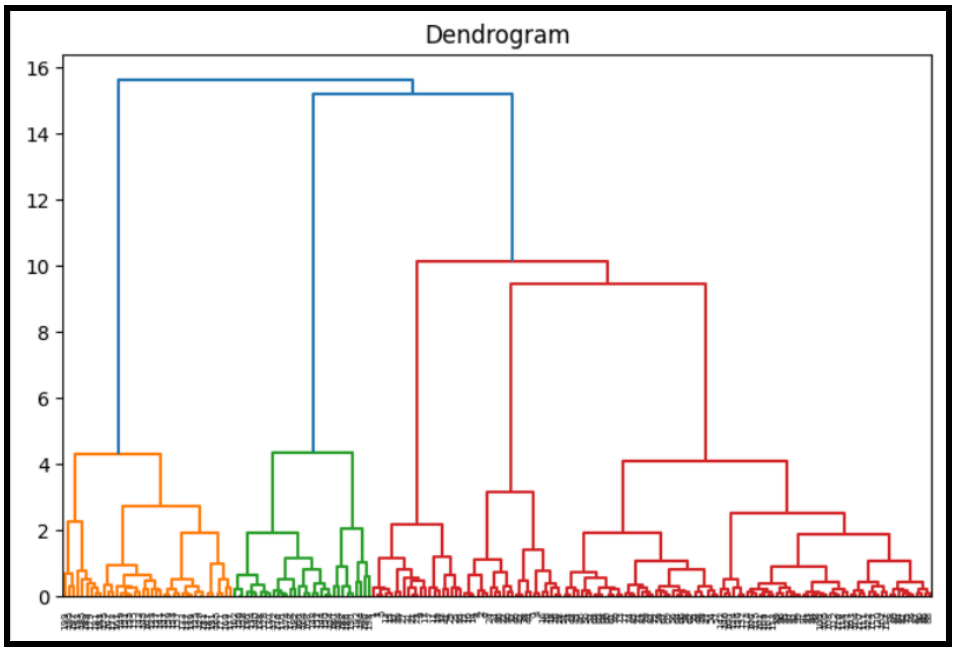
- Elbow graph




- Cluster visualization




- Dendrogram




**CONCLUSION**

In this experiment, K-Means and Hierarchical Clustering were successfully implemented.

The results show that:

- K-Means efficiently segments customers
- Hierarchical clustering provides better visualization
- Both techniques are useful for unsupervised learning

This experiment demonstrates clustering techniques used in real-world applications like customer segmentation.